MadeManifest – Inferred Module I/O and Data Flow Draft

OpenAI draft for Pascal Bourguignon

2026-03-22

Purpose of this draft

This document infers a plausible high-level data model and inter-module data flow from the following three PDFs:

- *Hybrid Architecture Specification*
- *System Flow*
- *One Pager – Deterministic Narrative Generation System*

It is not a statement of canon. It is a reconstruction intended to make the architecture more operational by spelling out likely JSON contracts, orchestration behavior, and missing questions.

The PDFs define responsibilities and boundaries, but mostly at the conceptual level. They do not define concrete API contracts. Therefore, all schemas below are **inferred** and should be validated with MadeManifest before implementation.

Executive summary

The specified architecture is **not** a purely linear pipeline in the strict stream-processing sense.

It is better described as:

- a deterministic generation pipeline for the main semantic flow: Input -> Profile -> Selection -> Blueprint -> Assembly -> Optional Smoothing -> Delivery
- with one sidecar / cross-cutting subsystem: Audit Layer
- and with at least one repository-style dependency: Content Service / Block Library
- plus a likely top-level application driver / orchestration layer, even though it is not explicitly named in the PDFs

In particular:

- *Profile Service*, *Selection Service*, and *Blueprint Service* look pipeline-like.
- *Assembly Service* does not merely consume previous output; it likely **drives** retrieval from the *Content Service* and may also emit detailed audit events.
- *Smoothing Service* is subordinate to the deterministic assembly result and must fall back to the deterministic output if validation fails.
- *Delivery Integration Layer* is more of an adapter / gateway than a pure transformation stage.
- *Audit Layer* is not in series: it runs alongside all steps.

Therefore the architecture is best modeled as a *coordinated pipeline with service calls*, not as a single simple chain of transforms.

System boundary inferred from the PDFs

The system starts at **validated input data** and ends at **delivery-ready outputs**.

Upstream of the system, and therefore out of scope:

- frontend / checkout
- user interaction
- raw input validation
- payment / subscription / order capture

Downstream of the system, and therefore out of scope:

- email scheduling
- sending
- retry policy
- external delivery execution

This strongly suggests that the internal boundary begins with something like a validated generation request and ends with a delivery payload, or with a stored generated output plus delivery-ready rendering.

Inferred top-level orchestration

The PDFs name modules, but do not explicitly name the process that coordinates them. In practice, one of the following must exist:

- an application service
- a job runner
- a workflow engine
- a backend endpoint handler
- a CLI command for deterministic generation

This orchestration layer probably:

1. accepts a generation request,

2. loads or computes profile data,
3. asks Selection for the relevant subset,
4. asks Blueprint for the structure for a given product / format,
5. asks Assembly to build the narrative, with Assembly itself querying the block library,
6. optionally invokes Smoothing,
7. persists output and trace,
8. optionally invokes Delivery Integration,
9. returns identifiers and artifacts.

Without such an orchestrator, the specified modules cannot be combined cleanly.

Candidate top-level request / response shapes

Generation request

A plausible canonical request entering the backend boundary:

```
{
  "request_id": "req_2026_04_02_0001",
  "subject_id": "user_12345",
  "input_version": "validated-input-v1",
  "input_data": {
    "person": {
      "given_name": "Pascal",
      "family_name": "Bourguignon"
    },
    "birth": {
      "date": "1968-08-01",
      "time": "16:04:00",
      "timezone": "Europe/Paris",
      "location": {
        "name": "Paris, France",
        "latitude": 48.8566,
        "longitude": 2.3522
      }
    },
    "declared_context": {
      "language": "en",
      "product_request": "email_journey"
    }
  },
  "product": {
    "product_type": "email_journey",
    "product_id": "manifest_email_journey",
    "format_variant": "standard",
```

```
    "locale": "en",
    "length_profile": "medium"
  },
  "execution": {
    "profile_version": "profile-rules-v3",
    "selection_version": "selection-rules-v2",
    "blueprint_version": "blueprint-email-v5",
    "content_version": "blocks-v12",
    "assembly_version": "assembly-rules-v4",
    "smoothing_enabled": true,
    "smoothing_policy": "fallback-on-validation-failure"
  },
  "delivery": {
    "provider": "customer_io",
    "channel": "email",
    "send_immediately": false
  }
}
```

Notes:

- The actual $\text{input}_{\text{data}}$ may be narrower or broader.
- The version fields are not stated explicitly in the PDFs for every service, but some form of version pinning is implied everywhere.
- If the system is intended to be reproducible, a fully explicit execution context is highly desirable.

Generation response

```
{
  "request_id": "req_2026_04_02_0001",
  "generation_id": "gen_2026_04_02_0001",
  "status": "generated",
  "profile_id": "prof_abc123",
  "selection_id": "sel_def456",
  "blueprint_id": "bp_email_v5",
  "output_id": "out_xyz789",
  "delivery_payload_id": "dp_001",
  "audit_trace_id": "trace_001",
  "used_versions": {
    "profile_version": "profile-rules-v3",
    "selection_version": "selection-rules-v2",
    "blueprint_version": "blueprint-email-v5",
    "content_version": "blocks-v12",
    "assembly_version": "assembly-rules-v4",
    "smoothing_version": "smooth-v1"
  }
```

```
}
```

Inferred high-level data model

The PDFs imply at least the following conceptual entities:

- *Validated Input*
- *Profile*
- *Selection*
- *Blueprint*
- *Content Block*
- *Narrative Assembly*
- *Smoothed Narrative*
- *Delivery Payload*
- *Audit Trace*

A plausible object graph is:

- one *Generation Request*
  - produces one *Profile*
  - one *Selection* for a given product
  - one *Blueprint* for a given format
  - one or many *Assembled Output Units*
  - optionally one *Smoothed Output*
  - one *Delivery Payload*
  - one *Audit Trace*

Relation to the known MadeManifest PoC / Golden Case

Based on the PoC and prior project context, the *Profile* probably contains more than generic marketing variables. It likely includes deterministic variables derived from a birth/input record, such as some combination of:

- normalized birth metadata,
- planetary or ephemeris-derived values,
- Human Design activations,
- Gene Keys or related derived structures,
- profile- or chart-level summary variables,
- derived categorical labels used by selection rules.

This matters because the downstream modules appear to treat variables as *already computed meaning*, not as free text.

That suggests a profile JSON more like a structured semantic snapshot than a simple user attributes object.

Module-by-module inferred contracts

1. Profile Service

Role

Transforms validated input data into a structured, versioned profile containing all deterministically computed variables needed downstream.

Likely input

```
{
  "request_id": "req_...",
  "input_data": {
    "person": { "...": "..." },
    "birth": { "...": "..." },
    "preferences": { "...": "..." }
  },
  "profile_version": "profile-rules-v3"
}
```

Likely output

```
{
  "profile_id": "prof_abc123",
  "profile_version": "profile-rules-v3",
  "subject_id": "user_12345",
  "normalized_input": {
    "birth": {
      "utc_timestamp": "1968-08-01T14:04:00Z",
      "timezone": "Europe/Paris",
      "location": {
        "latitude": 48.8566,
        "longitude": 2.3522
      }
    }
  },
  "variables": {
    "astrology": {
      "sun": { "sign": "Leo", "degree": 8.1234 },
      "moon": { "sign": "Aries", "degree": 19.4321 }
    },
    "human_design": {
      "type": "Generator",
      "profile": "4/6",
      "authority": "Sacral",
      "defined_centers": ["Sacral", "G"],
```

```
      "channels": ["34-20"],
      "gates": {
        "personality_sun": { "gate": 10, "line": 4 },
        "design_earth": { "gate": 15, "line": 2 }
      }
    },
    "gene_keys": {
      "activation_sequence": {
        "life_work": 10,
        "evolution": 15
      }
    },
    "derived_labels": [
      "self-direction",
      "creative-flow"
    ]
  },
  "traceability": {
    "calculation_inputs_hash": "sha256:...",
    "ephemeris_version": "2.10.03",
    "canon_version": "canon-2026-04"
  }
}
```


Comments

- Exact nested field names are unknown.
- The key architectural fact is that *all meaning used downstream should already exist here as deterministic structured variables.*
- Since the Profile is the stated source of truth for variables, downstream services should not recompute semantic meaning.

Likely persistence

- stored as a versioned JSON document
- probably keyed by $\text{subject}_{\text{id}}$ + $\text{profile}_{\text{version}}$ + canonical input hash

Risks

- If the profile schema is not explicit, Selection will become ambiguous.
- If variables are too raw, meaning leaks into Selection or Assembly.
- If variables are too derived, traceability may be lost unless lineage is stored.

2. Selection Service

Role

Chooses a deterministic subset of profile variables for a given product or output.

Likely input

```
{
  "profile_id": "prof_abc123",
  "profile_version": "profile-rules-v3",
  "product": {
    "product_type": "email_journey",
    "product_id": "manifest_email_journey",
    "format_variant": "standard",
    "locale": "en"
  },
  "selection_version": "selection-rules-v2"
}
```

Likely output

```
{
  "selection_id": "sel_def456",
  "selection_version": "selection-rules-v2",
  "profile_id": "prof_abc123",
  "product_id": "manifest_email_journey",
  "selected_variables": [
    {
      "variable_key": "human_design.type",
      "value_ref": "/variables/human_design/type",
      "priority": 100,
      "role": "core"
    },
    {
      "variable_key": "human_design.profile",
      "value_ref": "/variables/human_design/profile",
      "priority": 95,
      "role": "core"
    },
    {
      "variable_key": "gene_keys.activation_sequence.life_work",
      "value_ref": "/variables/gene_keys/activation_sequence/life_work",
      "priority": 80,
      "role": "supporting"
    }
  ],
```

```
  "selection_summary": {
    "variable_count": 12,
    "product_focus": "introductory_journey"
  }
}
```

Comments

- Selection probably returns references to profile variables, not copies.
- A *slot* or *theme* assignment may also happen here, but the PDFs do not assign that explicitly.
- Since Selection must "prioritize relevance" and "limit number of variables per output", it probably uses ranked rules, exclusion rules, and caps.

Risks

- If Selection returns only keys with no rationale, debugging becomes difficult.
- If Selection embeds text labels that are not in the Profile, it starts to interpret meaning.
- The distinction between *selected variables* and *section assignment* must be clarified: is section placement part of Selection or Assembly?

3. Blueprint Service

Role

Provides a static, versioned structure for a given output type.

Likely input

```
{
  "product": {
    "product_type": "email_journey",
    "product_id": "manifest_email_journey",
    "format_variant": "standard",
    "locale": "en"
  },
  "blueprint_version": "blueprint-email-v5"
}
```

Likely output

```
{
  "blueprint_id": "bp_email_v5",
  "blueprint_version": "blueprint-email-v5",
  "product_id": "manifest_email_journey",
  "output_type": "email_journey",
```

```
  "structure": {
    "units": [
      {
        "unit_id": "email_01",
        "unit_type": "email",
        "sections": [
          {
            "section_id": "opening",
            "slot": "intro",
            "required": true,
            "max_blocks": 2
          },
          {
            "section_id": "core_pattern",
            "slot": "theme_primary",
            "required": true,
            "max_blocks": 3
          },
          {
            "section_id": "closing",
            "slot": "cta_soft",
            "required": true,
            "max_blocks": 1
          }
        ]
      }
    ]
  }
}
```

Comments

- For reports, this might instead produce chapters, sections, subsections.
- Blueprint likely contains more than section order; it may define allowed slots, mandatory sections, repetition constraints, and target lengths.
- It is probably static data rather than executable logic.

Risks

- If the blueprint does not define slots precisely, Assembly will absorb too much policy.
- If product-specific sequencing is hidden elsewhere, determinism weakens.
- Need to know whether cadence metadata for email journeys is in scope or external.

4. Content Service / Block Library

Role

Stores reusable content blocks linked to variables and retrievable by variable and slot.

Likely input patterns

This service is probably not driven as a pipeline stage. It is more likely queried by Assembly.

Possible query:

```
{
  "content_version": "blocks-v12",
  "locale": "en",
  "variable_key": "human_design.type",
  "slot": "intro",
  "context": {
    "product_id": "manifest_email_journey",
    "section_id": "opening"
  }
}
```

Likely output

```
{
  "matches": [
    {
      "block_id": "blk_1001",
      "content_version": "blocks-v12",
      "variable_key": "human_design.type",
      "slot": "intro",
      "variant_id": "v1",
      "tone": "neutral-warm",
      "text_template": "Your energy tends to move best when ...",
      "constraints": {
        "products": ["manifest_email_journey"],
        "locales": ["en"]
      }
    },
    {
      "block_id": "blk_1002",
      "content_version": "blocks-v12",
      "variable_key": "human_design.type",
      "slot": "intro",
      "variant_id": "v2",
```

```
      "tone": "direct",
      "text_template": "At your core, you are designed to ...",
      "constraints": {
        "products": ["manifest_email_journey"],
        "locales": ["en"]
      }
    }
  ]
}
```

Comments

- Since blocks are said to be linked to variables and retrievable by variable and slot, some join key like `(variable_key, slot, locale, content_version)` almost certainly exists.
- Variants appear to be pre-authored, not generated at runtime.
- Blocks may be plain text, or parameterized templates with placeholders.

Risks

- Need to know whether blocks bind to single variables only, or to composite conditions.
- Need to know how locale, tone, audience, and product constraints are encoded.
- Need to know whether transition snippets live here too, or are separate assets.

5. Assembly Service

Role

This is the semantic composition core. It likely consumes:

- the Selection result,
- the Blueprint,
- profile values referenced by Selection,
- and blocks fetched from Content Service.

It then produces the deterministic narrative output.

Likely input

```
{
  "profile_id": "prof_abc123",
  "selection_id": "sel_def456",
  "blueprint_id": "bp_email_v5",
  "assembly_version": "assembly-rules-v4",
```

```
  "locale": "en"
}
```

Internally, it likely resolves:

- selected variable values from Profile
- block candidates from Content Service
- deterministic variant selection rules

Likely output

```
{
  "output_id": "out_xyz789",
  "assembly_version": "assembly-rules-v4",
  "product_id": "manifest_email_journey",
  "output_type": "email_journey",
  "units": [
    {
      "unit_id": "email_01",
      "sections": [
        {
          "section_id": "opening",
          "selected_variable_keys": ["human_design.type"],
          "block_refs": [
            { "block_id": "blk_1002", "variant_id": "v2" }
          ],
          "rendered_text": "At your core, you are designed to ..."
        },
        {
          "section_id": "core_pattern",
          "selected_variable_keys": [
            "human_design.profile",
            "gene_keys.activation_sequence.life_work"
          ],
          "block_refs": [
            { "block_id": "blk_2101", "variant_id": "v1" },
            { "block_id": "blk_3105", "variant_id": "v3" }
          ],
          "transition_refs": [
            { "block_id": "tr_004", "variant_id": "v1" }
          ],
          "rendered_text": "..."
        }
      ],
      "assembled_text": "full email body here"
    }
  ],
```

```
  "deterministic_output_hash": "sha256:..."
}
```

Comments

- Assembly probably acts more like a *driver* than a passive transform: it likely calls Content Service repeatedly.
- The phrase "map variables to sections" is important. This mapping may be performed here rather than in Selection.
- "Select deterministic variants" implies the existence of deterministic selection rules over candidate variants. Those rules are not specified in the PDFs.

Deterministic variant selection

A likely rule family:

- stable ordering of candidate variants
- selection by hash of: $\text{profile}_{\text{id}} + \text{variable}_{\text{key}} + \text{section}_{\text{id}} + \text{content}_{\text{version}} + \text{assembly}_{\text{version}}$
- or static first-match policy
- or explicit variant routing table

This must be clarified, because "multiple variants" plus "deterministic output" requires an explicit tie-break policy.

Risks

- This module is underspecified in the PDFs relative to its centrality.
- It can easily become the place where hidden semantics accumulate.
- Need to know whether transitions are authored blocks, generated glue, or blueprint assets.

6. Smoothing Service (optional)

Role

Accepts the deterministic assembled output and returns a readability-improved version without semantic change.

Likely input

```
{
  "output_id": "out_xyz789",
  "original_output": {
    "units": [
      {
        "unit_id": "email_01",
```

```
        "assembled_text": "..."
      }
    ]
  },
  "smoothing_policy": {
    "enabled": true,
    "must_preserve_meaning": true,
    "fallback_to_original_on_failure": true
  }
}
```

Likely output

```
{
  "smoothed_output_id": "sout_001",
  "source_output_id": "out_xyz789",
  "status": "accepted",
  "units": [
    {
      "unit_id": "email_01",
      "smoothed_text": "..."
    }
  ],
  "validation": {
    "passed": true,
    "validator_version": "semantic-guard-v1"
  }
}
```

or, on failure:

```
{
  "smoothed_output_id": null,
  "source_output_id": "out_xyz789",
  "status": "rejected_fallback_to_original",
  "validation": {
    "passed": false,
    "reasons": ["meaning_change_detected"]
  }
}
```

Comments

- This service may use AI, but the PDFs explicitly deny it authority over meaning.
- Therefore a validation stage is logically required, even if not named as a separate module.

- Either:
  - validation is inside Smoothing Service, or
  - there is an unstated *Smoothing Validator* submodule.

Risks

- How is "meaning unchanged" validated?
- If validation is heuristic, this may weaken determinism.
- Need to know whether smoothed output becomes authoritative, or whether both are persisted.

7. Delivery Integration Layer

Role

Transforms the final generated artifact into provider-specific delivery payloads and records external references.

Likely input

```
{
  "output_id": "out_xyz789",
  "preferred_rendering": "smoothed_if_accepted_else_original",
  "delivery": {
    "provider": "customer_io",
    "channel": "email"
  }
}
```

Likely output

```
{
  "delivery_payload_id": "dp_001",
  "provider": "customer_io",
  "channel": "email",
  "payload": {
    "campaign_key": "manifest_email_journey",
    "messages": [
      {
        "message_key": "email_01",
        "subject": "Your energetic pattern begins here",
        "body_html": "<p>...</p>",
        "body_text": "..."
      }
    ]
  },
  "external_references": [
```

```
    {
      "provider": "customer_io",
      "provider_reference": "cio_campaign_123"
    }
  ]
}
```

Comments

- The PDFs say this layer should not schedule or retry.
- Therefore this layer is an adapter to provider formats, not a full messaging engine.
- "Track delivery references" implies a mapping table from internal output units to external identifiers.

Risks

- Need to know whether rendering to HTML is here or earlier.
- Need to know whether report PDFs are also produced here or by another renderer.
- Need to know whether one internal output can map to several provider payloads.

8. External Delivery System

Role

Out of scope operational provider, e.g. ESP or report delivery platform.

Not authoritative

The PDFs are clear that this is execution-only and not source of truth.

Likely exchanged payload

Whatever provider-specific payload the Delivery Integration Layer emits.

9. Audit Layer

Role

Cross-cutting trace system that records every important input, decision, version, and artifact reference.

Likely event model

Rather than one monolithic document written only at the end, the architecture suggests an append-only trace of events such as:

```json
{
  "trace_id": "trace_001",
  "events": [
    {
      "step": "profile_generated",
      "profile_id": "prof_abc123",
      "profile_version": "profile-rules-v3",
      "input_hash": "sha256:..."
    },
    {
      "step": "variables_selected",
      "selection_id": "sel_def456",
      "selected_variable_keys": [
        "human_design.type",
        "human_design.profile"
      ]
    },
    {
      "step": "blueprint_loaded",
      "blueprint_id": "bp_email_v5"
    },
    {
      "step": "block_selected",
      "section_id": "opening",
      "variable_key": "human_design.type",
      "block_id": "blk_1002",
      "variant_id": "v2"
    },
    {
      "step": "output_assembled",
      "output_id": "out_xyz789",
      "output_hash": "sha256:..."
    },
    {
      "step": "delivery_payload_created",
      "delivery_payload_id": "dp_001"
    }
  ]
}
```

Comments

- For reproducibility, the audit trace should capture:
  - all input hashes
  - all version identifiers
  - all selected variable keys

  - all block / variant IDs
  - all deterministic tie-break decisions
  - smoothing acceptance / rejection
  - external provider references

Risks

- If audit stores only final summaries, reconstruction will fail.
- If audit is mutable, reproducibility and dispute resolution weaken.

Inferred data dependencies matrix

Module	Reads	Writes	Likely pe
Profile Service	validated input, canon	profile	profile st
Selection Service	profile, product rules	selection	selection
Blueprint Service	product/format request, blueprint defs	blueprint resolution	blueprint
Content Service	content library data	block candidates	content r
Assembly Service	profile, selection, blueprint, content blocks	deterministic output	output st
Smoothing Service	deterministic output	smoothed output or rejection	output st
Delivery Integration	final output, delivery config	delivery payload, provider refs	delivery r
Audit Layer	events from all modules	trace records	audit sto

Inferred control-flow shape

Not purely serial

At least the following call graph is plausible:

```
Generation Driver
  -> Profile Service
  -> Selection Service
  -> Blueprint Service
  -> Assembly Service
       -> Content Service (repeated lookups)
  -> Smoothing Service (optional)
  -> Delivery Integration Layer
  -> Audit Layer (parallel events throughout)
```

So the chain shown in the PDFs is correct at the headline level, but operationally:

- *Assembly Service* is a driver of *Content Service*
- *Generation Driver* is the driver of all major steps
- *Audit Layer* is orthogonal
- *Delivery Integration Layer* is a gateway, not just a transformation stage

Possible alternative orchestration

Another implementation option would be event-driven orchestration:

- profile-generated event
- selection-computed event
- blueprint-resolved event
- assembly-requested event
- output-generated event
- smoothing-requested event
- delivery-payload-created event

But the PDFs do not suggest asynchronous architecture explicitly. Their tone suggests a deterministic request-driven workflow, likely synchronous or job-based.

Missing technical details inferred from the gaps

The PDFs leave several implementation-critical items unspecified:

Identity and addressing

- How are users / subjects identified?
- Is generation based on a user id, an order id, or a one-shot input payload?
- Can the same subject have multiple profiles under different canon versions?

Versioning

- Is every service versioned independently?
- Is there one global canon version?
- Are content, blueprint, and selection rules versioned separately?
- What is the regeneration key? input hash + product + versions?

Schema ownership

- What is the canonical profile JSON schema?
- What are the variable naming conventions?
- Are nested structures allowed as selection targets, or only flattened keys?
- Are there typed variables, e.g. scalar, enum, range, composite, list?

Section mapping

- Does Selection choose variables only, or does it also assign them to sections?
- Does Assembly map variables to sections via rules?
- Where do narrative priorities live?

Content model

- Are blocks single-variable only, or can one block depend on several variables?
- Are variants purely stylistic, or semantically distinct but equivalent?
- How are transitions represented?
- Are placeholders permitted inside blocks?

Smoothing validation

- What exact validator decides that meaning has not changed?
- Is smoothing reproducible?
- Is the smoothed output versioned independently?
- Is the original deterministic output always stored alongside the smoothed one?

Rendering and delivery

- Who generates email subject lines?
- Who renders HTML and plain text?
- Who renders report PDF / DOCX / HTML?
- Are delivery payloads stored in canonical normalized form as well as provider form?

Persistence and audit

- What stores are authoritative?
- Is the audit append-only?
- Can any output be reconstructed exactly from trace + versions + repositories?

Suggested normalized JSON entities

Below is a concise candidate entity list that could be adopted.

Profile

- `profile_id`
- `profile_version`
- `subject_id`
- `normalized_input`
- `variables`
- `traceability`

Selection

- selection_id
- selection_version
- profile_id
- product_id
- selected_variables[]
- selection_summary

Blueprint

- blueprint_id
- blueprint_version
- product_id
- output_type
- structure

Content block

- block_id
- content_version
- variable_key
- slot
- variant_id
- locale
- constraints
- text_template

Output

- output_id
- assembly_version
- profile_id
- selection_id
- blueprint_id
- units[]
- deterministic_output_hash

Smoothed output

- smoothed_output_id
- source_output_id
- validation
- units[]

Delivery payload

- `delivery_payload_id`
- `output_id`
- `provider`
- `channel`
- `payload`
- `external_references[]`

Audit trace

- `trace_id`
- `request_id`
- `events[]`

My current best interpretation of the architecture

The architecture is a *deterministic semantic assembly backend*.

It is **not** yet a full platform specification.

Concretely:

- The semantic core is well enough described at the concept level.
- The integration contracts are not yet adequately specified for implementation.
- The engine boundary can be made clean only if MadeManifest confirms:
  - the input schema,
  - the profile schema,
  - selection semantics,
  - blueprint schema,
  - content block retrieval model,
  - assembly rules,
  - smoothing validation,
  - delivery payload expectations.

Questions for MadeManifest

Below is the question list to carry into the kickoff or canon-package review.

Scope and boundary

1. Can you confirm the exact boundary of the engine implementation: does it end at deterministic assembled output, at validated smoothing, or at provider-specific delivery payload generation?

2. Is *Delivery Integration Layer* part of the engine implementation scope for this phase, or is only the deterministic generation core in scope?
3. Is frontend / checkout permanently out of scope for this engagement, or just out of scope for the first implementation phase?

Input contract

1. What is the canonical JSON schema for the validated input entering the system?
2. Which input fields are guaranteed to be validated upstream, and which invariants may the backend rely on without revalidation?
3. Is generation addressed by raw input payload, by subject id, by order id, or by another identifier?

Profile schema

1. What is the canonical schema of the generated Profile object?
2. Are variables addressed as hierarchical JSON paths, flattened keys, or both?
3. Which variables are considered first-class selection targets?
4. Are all downstream semantics expected to come exclusively from the Profile, with no reinterpretation in later modules?

Selection semantics

1. Does Selection only choose variables, or does it also assign them to sections / slots / narrative roles?
2. What deterministic policy governs prioritization and cap limits when too many variables are candidates?
3. Should Selection return variable references, copied values, scored candidates, or a richer explanation object?

Blueprint schema

1. What is the canonical blueprint representation? Sections only, or also slots, length targets, repetition rules, and narrative arcs?
2. For email journeys, does the blueprint include cadence metadata, or is cadence entirely external to the engine boundary?
3. Are blueprints global per product, or can they vary by locale, customer segment, or profile class?

Content block model

1. What is the canonical content block schema?
2. Are blocks keyed by single variable + slot only, or can they depend on compound conditions involving multiple variables?
3. How are content variants selected deterministically?
4. Are transitions stored as ordinary blocks, as blueprint elements, or generated by assembly rules?
5. Are blocks plain text only, or may they contain placeholders / templates?

Assembly rules

1. What exact data does Assembly receive as input: full Profile, Selection result, Blueprint, content references, or all of the above?
2. Where is variable-to-section mapping defined: Selection, Blueprint, or Assembly?
3. What deterministic tie-break rules are used when several valid blocks or variants exist for the same slot?
4. Should Assembly emit a structured intermediate representation in addition to final text, so outputs remain decomposable and auditable?

Smoothing

1. Is Smoothing in scope for phase 1, or should the first implementation deliver only the deterministic assembled output?
2. What validator decides whether smoothing preserved meaning?
3. If smoothing passes, does the smoothed output become the primary delivered output, while the deterministic one remains the canonical source of truth?
4. Is smoothing required to be reproducible, and if so, how is reproducibility enforced?

Delivery and rendering

1. What exact delivery-ready payloads are expected for the first supported output types?
2. Who owns rendering into HTML / plain text / report formats?
3. Are delivery provider adapters expected now, or should the engine output an internal normalized representation only?
4. What external references must be stored after handoff to delivery systems?

Persistence, audit, and regeneration

1. Which stores are authoritative for: profile, output, delivery references, and audit trace?
2. Is the audit log expected to be append-only?
3. What minimum data must be captured to reconstruct an output exactly?
4. What combination of input hash, product id, and version ids defines output identity?
5. On regeneration after canon or content changes, should prior outputs remain frozen and separately addressable?

Operational mode

1. Should the implementation be exposed as: a library, a CLI, an internal API, or some combination of the three?
2. Is the expected workflow synchronous request/response, queued job processing, or batch generation?

Closing note

The module decomposition in the PDFs is conceptually clear and coherent, but it is still one level above implementable contracts. The main hidden complexity lies in the places where the PDFs remain abstract:

- exact JSON schemas,
- deterministic tie-break rules,
- ownership of section mapping,
- content block addressing,
- smoothing validation,
- and delivery payload ownership.

Those are the points that should be pinned before implementation begins.